Reimplementing Core Components of Llama 2

Project Report

Phan Hoang Dung – 23020346
Ngo Quang Dung – 23020344
Truong Trong Duc – 23020360

1 Overview

This assignment requires students to independently reimplement the core components of the Llama 2 language model-one of the leading open-source large language models (LLMs) today. Through this process, students will gain a solid understanding of modern Transformer architecture and the training pipeline for text classification models. The assignment uses `stories42M.pt`, a compact Llama 2 model with 42 million parameters, consisting of 8 Transformer layers, pre-trained on the TinyStories dataset.

2 Implementation

2.1 RMSNorm (`llama.py`)

Root Mean Square normalization is computed by scaling x with the inverse square root of the mean of the squared elements. The implementation uses `x.pow(2).mean(-1, keepdim=True)` combined with `torch.rsqrt` for efficient normalization. A small constant `self.eps` is added to avoid division by zero, and the result is scaled by the learnable parameter `self.weight`.

2.2 Rotary Position Embedding (`rope.py`)

RoPE encodes relative positional information by rotating pairs of elements in the query/key vectors with an angle proportional to the token position. The implementation separates real and imaginary parts using `unbind`, computes the rotation angles via `torch.outer`, broadcasts cos and sin to the correct shape, and applies the complex rotation:

$$q_r \cdot \cos - q_i \cdot \sin \quad \text{and} \quad q_r \cdot \sin + q_i \cdot \cos$$

2.3 Scaled Dot-Product Attention (`llama.py`)

Attention scores are computed using QK^T, scaled by $1/\sqrt{d_k}$, followed by softmax and dropout. The result is then multiplied by V to obtain the context vector. All heads are processed in parallel.

2.4 Transformer Block (`llama.py`)

A pre-norm architecture is used with two sub-layers:

$$h = x + \text{Attention}(\text{RMSNorm}(x))$$

followed by:

$$\text{out} = h + \text{FFN}(\text{RMSNorm}(h))$$

Residual connections in both steps help stabilize gradient flow in deep networks.

2.5 Text Continuation (`llama.py`)

With temperature = 0.0, greedy decoding (`argmax`) is used.

With temperature > 0.0, logits are divided by the temperature before applying softmax, and sampling is performed using `torch.multinomial`. Generated tokens are appended to the sequence iteratively until reaching `max_new_tokens`.

2.6 Classification Head (`classifier.py`)

The model calls `self.llama(input_ids)` to obtain hidden states, extracts the last position `[:, -1, :]` as the sentence representation, applies dropout, then feeds it through a Linear layer dim → num_labels, followed by log-softmax.

2.7 AdamW (`optimizer.py`)

First and second moments are updated using β_1 and β_2. Bias correction is applied when `correct_bias = True` to compensate for zero initialization. Parameters are updated using:

$$\theta \leftarrow \theta - \text{step_size} \times \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon}$$

Then, decoupled weight decay is applied separately:

$$\theta \leftarrow \theta - \text{lr} \times \lambda \times \theta$$

This is the key difference compared to standard Adam.

3 Experiments

3.1 Environment Setup

```
bash setup.sh
conda activate llama_hw
```

3.2 Component Testing

```
# Test RoPE
python rope_test.py
# Test AdamW optimizer
python optimizer_test.py
# Test full Llama model
python sanity_check.py
```

3.3 Text Continuation

```
python run_llama.py \
    --option generate \
    --pretrained_model_path stories42M.pt \
    --generated_sentence_low_temp_out generated-sentence-temp-0.txt \
    --generated_sentence_high_temp_out generated-sentence-temp-1.txt
```

3.4 Zero-shot Prompting

```
# SST
python run_llama.py \
    --option prompt \
    --train data/sst-train.txt \
    --dev   data/sst-dev.txt \
    --test  data/sst-test.txt \
    --label-names data/sst-label-mapping.json \
    --pretrained_model_path stories42M.pt \
    --dev_out  sst-dev-prompting-output.txt \
    --test_out sst-test-prompting-output.txt
# CFIMDB
python run_llama.py \
    --option prompt \
    --train data/cfimdb-train.txt \
    --dev   data/cfimdb-dev.txt \
    --test  data/cfimdb-test.txt \
    --label-names data/cfimdb-label-mapping.json \
    --pretrained_model_path stories42M.pt \
    --dev_out  cfimdb-dev-prompting-output.txt \
    --test_out cfimdb-test-prompting-output.txt
```

3.5 Fine-tuning

Run with default parameters:

```
# SST - default hyperparameters (lr=2e-5, epochs=5, dropout=0.3, batch=64)
python run_llama.py \
    --option finetune \
    --train data/sst-train.txt \
    --dev   data/sst-dev.txt \
    --test  data/sst-test.txt \
    --pretrained_model_path stories42M.pt \
    --batch_size 64 \
    --dev_out  sst-dev-finetuning-output.txt \
    --test_out sst-test-finetuning-output.txt \
    --use_gpu
# CFIMDB - default hyperparameters (lr=2e-5, epochs=5, dropout=0.3, batch=8)
python run_llama.py \
    --option finetune \
    --train data/cfimdb-train.txt \
    --dev   data/cfimdb-dev.txt \
    --test  data/cfimdb-test.txt \
    --pretrained_model_path stories42M.pt \
    --batch_size 8 \
    --dev_out  cfimdb-dev-finetuning-output.txt \
    --test_out cfimdb-test-finetuning-output.txt \
    --use_gpu
```

Run full grid search:

```
# SST - save result at hyperparam_results_sst.csv
python run_hyperparam_search.py \
    --dataset sst \
    --train data/sst-train.txt \
    --dev   data/sst-dev.txt \
    --test  data/sst-test.txt \
    --pretrained_model_path stories42M.pt \
    --use_gpu
# CFIMDB - save result at hyperparam_results_cfimdb.csv
python run_hyperparam_search.py \
    --dataset cfimdb \
```

```
    --train data/cfimdb-train.txt \
    --dev   data/cfimdb-dev.txt \
    --test  data/cfimdb-test.txt \
    --pretrained_model_path stories42M.pt \
    --use_gpu
```

4 Experimental Results

4.1 Text Continuation

With temperature = 0.0 (greedy decoding), the model always selects the token with the highest probability, resulting in deterministic outputs. The generated sentences are coherent and repeatable across runs.

With temperature = 1.0 (sampling), the model samples from the full probability distribution, producing more diverse outputs.

In both cases, the generated English is grammatically correct, though the content often deviates from the original prompt because the model was pre-trained only on children's stories and lacks context about action movies. Results are saved in generated-sentence-temp-0.txt and generated-sentence-temp-1.txt.

4.2 Zero-shot Prompting

As expected, performance is quite poor on both datasets. Since the model was pre-trained on children's stories, it lacks semantic understanding of movie reviews, making predictions nearly random. This serves as a baseline for comparison with fine-tuning.

Dataset	**Dev Acc**	**Test Acc**
SST-5	0.215	0.223
CFIMDB	0.502	0.213

Table 1: Zero-shot Prompting Results

4.3 Fine-tuing

We first evaluated the model using default hyperparameters for both datasets:

- **SST-5:** lr = 2e-5, epochs = 5, dropout = 0.3, batch = 64.
- **CFIMDB:** lr = 2e-5, epochs = 5, dropout = 0.3, batch = 8.

Dataset	**Dev Acc**	**Test Acc**
SST-5	0.421	0.402
CFIMDB	0.857	0.469

Table 2: Fine-tuning Results (Default Parameters)

And then We performed a grid search over 12 hyperparameter combinations:

- **Learning Rate:** $\{1\text{e-}5, 2\text{e-}5, 5\text{e-}5\}$
- **Epochs:** $\{5, 10\}$
- **Dropout:** $\{0.1, 0.3\}$

Dataset	lr	Epochs	Dropout	Dev Acc	Test Acc
SST-5	1e-5	10	0.3	0.4223	0.4149
CFIMDB	1e-5	10	0.1	0.9143	0.5164

Table 3: Fine-tuning Results (Best Hyperparameters)

SST-5 Results (batch size 64): The best configuration achieved a dev accuracy of 0.4223 and test accuracy of 0.4149 with $\text{lr} = \text{1e-5}, \text{epochs} = 10, \text{dropout} = 0.3$. Overall, performance across configurations was relatively consistent, ranging from 0.396 to 0.422 on the dev set. A lower learning rate combined with more epochs yields the most stable results. While the improvement over zero-shot is significant, the 5-class classification task remains challenging for this model size.

CFIMDB Results (batch size 8): The best configuration achieved a high dev accuracy of 0.9143 ($\text{lr} = \text{1e-5}, \text{epochs} = 10, \text{dropout} = 0.1$). However, test accuracy only reached 0.5164, indicating severe overfitting. The model performs exceptionally well on the small dev set but fails to generalize to the test set. This is likely due to the small dataset size and low batch size, making the model more prone to overfitting compared to SST-5.

5 Conclusion

This assignment helped the team gain a deep understanding of the architecture and mechanisms of the Llama 2 language model by implementing its core components from scratch. These components include RMSNorm, Rotary Position Embedding, Scaled Dot-Product Attention, Transformer Blocks, text generation mechanisms, and the AdamW optimizer - all of which passed the required unit tests.

The three experiments, conducted in increasing order of complexity, clearly demonstrate the differences between approaches. Text generation confirms that the model functions correctly after loading pre-trained weights. Zero-shot prompting produces near-random results, showing that even a powerful model cannot perform well on specialized tasks without adaptation. Fine-tuning addresses this limitation and significantly improves performance, especially on CFIMDB with a dev accuracy of 0.9143.

However, the results also reveal a key limitation: the large gap between dev and test accuracy on CFIMDB indicates severe overfitting when training on small datasets with low batch sizes. On SST-5, accuracy plateaus around 0.42, reflecting the inherent difficulty of 5-class classification with a relatively small model.

From this assignment, two important practical lessons emerge:

- A small learning rate (1e-5) combined with more training epochs provides the most stable fine-tuning strategy for large language models.
- Implementing and testing individual components independently is an effective approach for building and debugging complex deep learning systems.